Tooling-Stack Verification — Operator-Supplied Third-Party AI Transcript

Field	Value
Revision	1
Created	2026-07-22T21:30:00Z
Last modified	2026-07-22T21:30:00Z
Status	COMPLETE — every named artifact verified or honestly UNKNOWN
Author	(T1/main - claude4) BG-TOOLING-VERIFY subagent, Fable, §11.4.182
Input under verification	`inputs/OPERATOR_SUPPLIED_TOOLING_MATERIAL_20260722.txt` (798 lines — a third-party AI transcript recommending an anti-bluff stack for Claude Code)
Builds on	`ROOT_CAUSE_ANALYSIS.md` (PC-1..PC-9) · `EXTERNAL_RESEARCH.md` + `SOURCES.md` (S1–S76) · `CROSS_PROJECT_CLAUDE_TOOLKIT.md` · `CROSS_PROJECT_HELIX_CODE.md` · `solutions/README.md` (SOL-01..SOL-10)
Verification date	2026-07-22 (all access dates below are 2026-07-22 unless stated)
Classification	universal (§11.4.17) — findings feed the Phase 3 solutions agent

. Framing and the one-sentence verdict

The operator-supplied material is **claims by another AI, not verified fact** — and it is itself a specimen of the pattern this research effort studies: confident, specific, immediately actionable, unverified. The verification verdict, in one sentence:

> **The transcript is roughly two-thirds real and one-third confabulated — every named package and repo except one dead link EXISTS, and two of them (fablize; the hooks-count claim) are surprisingly accurate — but the transcript fabricates specific commands (`npx truthguard audit` , `truthguard pre` / `post`), fabricates a config file and a log path, ships a hook example built on an environment variable that does not exist (a silent false-negative guard — the exact §11.4.201 shape), recommends one DEPRECATED package and one package whose source repository has been DELETED, and presents 2-star dead weekend projects side-by-side with 870-star maintained tools with identical confidence.** Following it verbatim would produce exactly the false-success failure it promises to cure. Cherry-picked with verification, it contains two genuinely valuable seam mechanisms.

Nothing was installed. No settings, hooks, agents, or governance files were modified. This document is the only write.

. Method + instrument certification (control needles, § . . ( ) (b))

All existence checks ran through two instrument paths, each certified with a control needle BEFORE any absence was reported:

Needle	Path	Result
Known-present npm package `express`	`curl https://registry.npmjs.org/express`	HTTP 200, name/latest/modified parsed (`5.2.1` , modified 2026-07-14)
Known-present GitHub repo `anthropics/claude-code`	`curl https://api.github.com/repos/...`	HTTP 200, 138,685 stars, pushed 2026-07-21
Known-ABSENT GitHub repo (invented name)	same path	HTTP 404 `{"message":"Not Found"}` — the absent shape is visible and distinct

Additional instrument facts honestly recorded:

- GitHub API **rate-limited (HTTP 403)** partway through the run. One query (`zed-industries/agent-client-protocol`) returned empty fields that were **NOT read as absence**; the raw re-check proved 403, and that repo's existence is instead evidenced via the ACP ecosystem sources in §4.9 (JetBrains official ACP pages + the live npm adapter package).
- Per-query needles were used for content greps (e.g. before reporting `audit` absent from the TruthGuard CLI, the literal `install` was counted through the same grep path: 7 hits).
- `find -newermt` was not used anywhere (known-unreliable on this host).
- The four completed research docs were grepped for every tool name in the transcript (`truthguard|fablize|darcyegb|claude-pty-acp|stop-hallucination|mcp-postgres|ClayCheung|sergeeey|VERIFIED-REAL|Jenny|Karen`) — **zero hits** — so this verification duplicates nothing already covered.

. Verdict summary table

#	Artifact (as named in transcript)	Exists?	Alive?	Does what is claimed?	Supply-chain risk	Verdi
1	`truthguard` (npm) + `spyrae/truthguard` (GitHub) + brew tap	**EXISTS**	**DEAD** — 1 release (0.1.0, 2026-03-09), 0 updates, 2 stars, 14 downloads/month	Core checksum/exit-code claims REAL in source; `audit` command + config file FABRICATED	Moderate: 7 shell scripts + 237-line CLI run on every Bash/Write/Edit; fully auditable but unmaintained	**ADAP** not pa
2	`skills` CLI (`npx skills add`)	**EXISTS**	**ALIVE** — vercel-labs/skills, 87 versions, modified 2026-07-22, 51,903,010 dl/month	Yes — installs skills from GitHub repos	CLI itself credible (Vercel Labs); the risk is what it installs	N/A (c chann
3	`howardng97/claude-stop-hallucination` skill	**EXISTS**	**DEAD** — created AND last pushed 2026-04-29 (single day), 1 star, 0 forks	Yes — SKILL.md name `reality-check` , anti-sycophancy triggers as described	Low execution risk (prose skill) but it is arbitrary third-party INSTRUCTIONS injected into agent context	**REJE** ant +

#	Artifact (as named in transcript)	Exists?	Alive?	Does what is claimed?	Supply-chain risk	Verdi
4	Evidence tagging `[VERIFIED-REAL]` / `[VERIFIED-SYNTHETIC]` ("sergeeey repo")	**EXISTS** — `sergeeey/Claude-cod-top-2026`	Semi — 7 stars, pushed 2026-07-22; repo self-reports "dog-food evidence still growing (2 real runs)"	Yes — markers are a "hard rule in rules/integrity.md" of that repo	None (it is a CLAUDE.md text block)	**REJE prose boun by SC**
5	`darcyegb/Claude-CodeAgents` (Jenny, Karen, TaskCompletionValidator, +3)	**EXISTS**	**STALE** — 748 stars, 69 forks, last push 2026-05-23 (~2 months)	Yes — all 6 named agent files present (+1 extra) on branch `master`	**NO LICENSE** (all rights reserved — cloning into product workflow is legally unclean); content is prompts, not code	**REJE** ant to §11.4. + licer
6	`fivetaku/fablize` plugin	**EXISTS**	**ALIVE** — 870 stars, 117 forks, MIT, created 2026-06-14, pushed 2026-07-06	**Yes** — every specific claim verified in source (Stop-hook gate, goals.py decomposition, early-stop hook, honest-limits section, measurement protocol, tests)	Moderate: Python hooks + context packs run with our privileges; MIT, testable, auditable (48 files)	**ADAP** artifac stack

#	Artifact (as named in transcript)	Exists?	Alive?	Does what is claimed?	Supply-chain risk	Verdi
7	`anthropics/claude-plugins-official` + `mcp-server-dev` plugin	**EXISTS**	**ALIVE** — official Anthropic, 32,478 stars, pushed 2026-07-22	Yes — `mcp-server-dev:build-mcp-server` is in THIS session's live skill list (runtime evidence)	Low (official)	ADOP (alrea ively p
8	`@fabriciofs/mcp-postgres`	**EXISTS**	**DEAD-ish** — v1.0.1 (2026-01-22), repo pushed 2026-02-05, 1 star, 53 dl/month	Yes — all 5 claimed tool names present in README; but write-ops exist behind `READONLY=false`	Moderate — a DB-credentialed MCP server from a 1-star personal repo	**REJE proje** gres i ciple a
9	`@modelcontextprotocol/server-github`	**EXISTS**	**DEPRECATED** — npm latest carries "Package no longer supported"	Historically yes; recommending it in 2026-07 is stale advice	Deprecated dependency = unpatched surface	**REJE** `gh` C prese
10	`claude-pty-acp` (npm) + `ClayCheung/claude-pty-acp` (GitHub)	npm **EXISTS** (0.1.0, 2026-06-13); GitHub repo **404 — DELETED/ private**	Dead: 1 version, 40 dl/month, **source repo gone**	Unverifiable — the published tarball's source cannot be reviewed	**HIGH — unauditable published artifact; the transcript's `git clone` command fails**	**REJE**

#	Artifact (as named in transcript)	Exists?	Alive?	Does what is claimed?	Supply-chain risk	Verdi
11	ACP (Agent Client Protocol) + JetBrains support	**REAL**	ALIVE — official Zed adapter `@zed-industries/claude-code-acp` v0.16.2, 67,931 dl/month; JetBrains ACP registry live 2026-01	Yes — transcript's JetBrains claims (Dec 2025 support, Settings | Tools | AI Assistant | Agents, registry) accurate	Official-adapter path is credible; transcript recommended the wrong (dead) adapter	**REJE verifi ation** subag isolate cial ac or inte wante

[3]

. Platform-claims accuracy audit (the transcript's statements about Claude Code itself)

Checked against the live docs (`code.claude.com/docs/en/hooks` , fetched 2026-07-22) and this session's runtime:

Transcript claim	Verdict	Evidence
"The hook system has 30 hook events across session, prompt, tool, subagent, task, file, and notification phases"	**ACCURATE** (docs list ~30–31 events incl. SessionStart, UserPromptSubmit, PreToolUse, PostToolUse, PostToolUseFailure, SubagentStart/Stop, TaskCreated/Completed, Stop, FileChanged, Notification, …)	docs fetch 2026-07-22
Hook input arrives as JSON via stdin (Python example)	**ACCURATE** — "For command hooks, input arrives on stdin"	docs fetch
Hooks matcher-array settings schema (the "manual configuration" JSON block)	**ACCURATE** — matches the documented 3-level schema and matches this repo's live `.claude/settings.json`	docs fetch + local file
Step-2 hook block: `"PreToolUse": "npx truthguard pre $TOOL_NAME $TOOL_ARGS"` (flat string + env vars)	**FABRICATED TWICE** — (a) flat-string-per-event is not the schema; (b) `truthguard pre` / `truthguard post` subcommands DO NOT EXIST in the CLI (commands: `init`, `install`, `status`, `help`); (c) `$TOOL_NAME` / `$TOOL_ARGS` are not documented hook env vars	CLI source lines 40–45; docs fetch

Transcript claim	Verdict	Evidence
Force-push-block example using `$CLAUDE_TOOL_INPUT` piped through shell `grep`	**BROKEN AND DANGEROUS AS A PATTERN** — no `CLAUDE_TOOL_INPUT` env var is documented; the variable expands empty, the grep never matches, the hook never blocks anything → a guard that silently always-allows, the precise §11.4.201 false-negative shape. Separately, interpolating tool input through a shell string is an injection-adjacent anti-pattern even where such a variable exists — real hooks must read stdin JSON with a proper parser (as our §11.4.109 guards do)	docs fetch (env vars: `CLAUDE_PROJECT_DIR`, `CLAUDE_PLUGIN_ROOT`, `CLAUDE_PLUGIN_DATA`, `CLAUDE_EFFORT`, …)
"grep for [VERIFIED-REAL] in `.claude/last-session.log`"	**FABRICATED PATH** — no such file exists in Claude Code's layout (transcripts live under the config dir's `projects/*.jsonl`)	local layout
`npx truthguard init` "creates a truthguard.config.json"	**FABRICATED** — 0 occurrences of `truthguard.config` in the CLI; `init` writes hooks into `.claude/settings.json`; the repo's config artifact is `.truthguard.yml.example`	CLI source (needle: `install` = 7 hits, same grep path)
`npx truthguard audit` — "shows every edit that wasn't confirmed"	**FABRICATED** — 0 occurrences of `audit` in the CLI	CLI source, needled

Transcript claim	Verdict	Evidence
`/plugin marketplace add …` / `/plugin install …` flow	**ACCURATE** (real plugin flow; `fablize` README uses the identical commands)	fablize README + local plugin ecosystem
`/reload-plugins` command	`UNKNOWN:` not independently verified this session (low stakes — restart achieves the same)	—
`claude mcp add --transport http <name> <url>`	Consistent with the documented MCP CLI shape; not independently re-verified this session — `UNKNOWN:` at the letter-exact level, low stakes	—
JetBrains "native ACP support since December 2025", Settings | Tools | AI Assistant | Agents, agent registry	**ACCURATE** — JetBrains "Bring your own AI agent" (2025-12), ACP Agent Registry live (2026-01), official ACP docs pages	JetBrains blog + help pages, accessed 2026-07-22 (see Sources)
PostToolUse "Can Block? No (already executed)"	Materially accurate for the executed tool; nuance: PostToolUse output can still steer the agent (TruthGuard's phantom-edit script emits `decision: "block"` as feedback)	docs + TruthGuard source

4 1

. Per-artifact detail

. TruthGuard (`truthguard` npm / `spyrae/truthguard` / `spyrae/homebrew-truthguard`)

- **Exists:** npm 200 — v0.1.0, created 2026-03-09, **1 version ever**, description "Catches false claims from AI coding agents. Verifies actions were actually performed." GitHub 200 — 2 stars, 2 forks, last push 2026-03-09. Homebrew tap repo exists (1 star). Downloads: **14/month**.
- **Alive:** NO. Single-day release, zero commits in 4.5 months, a `docs/launch/` directory (devto.md, reddit.md, ru-chat.md) evidencing a launch that did not take off. Evidence class: registry JSON + repo metadata + full tree listing.
- **Does what is claimed:** the two load-bearing behavioral claims are REAL in source, read line by line: `scripts/pre-file-change.sh` stores a sha256 of the target file before `Write|Edit` (stdin JSON parsed with `jq`), and `scripts/check-file-change.sh` re-hashes after, logs `phantom-edit`, and emits `{decision:"block", reason:"…Do NOT claim the file was updated."}` when the checksum is unchanged — the "ghost edit" catcher is genuine and correctly built on stdin JSON, not env vars. `check-exit-code.sh`, `block-dangerous.sh` (blocks `--no-verify`, force-push), `pre-commit-tests.sh` all exist. It is also packaged as a proper Claude plugin (`.claude-plugin/plugin.json` + `hooks/hooks.json` using `${CLAUDE_PLUGIN_ROOT}`). What is NOT real: `truthguard audit` (fabricated by the transcript), `truthguard.config.json` (fabricated), `truthguard pre` / `post` subcommands (fabricated).
- **Would it work here:** the hooks.json shape is compatible with our settings schema. But global state under `~/.truthguard/` interacts poorly with the multi-alias `CLAUDE_CONFIG_DIR=.claude-<alias>` layout (per-alias sessions, one shared home-dir checksum store — cross-session checksum collisions between concurrent tracks/aliases writing the same paths are plausible; the store is keyed by file path hash only, no session id). No container requirement (§11.4.161 N/A).
- **Supply-chain:** small enough to audit exhaustively (this session read the load-bearing scripts in full); LICENSE file present but GitHub reports `NOASSERTION`. Unmaintained = adopting the package means adopting ownership of it.

- **Verdict: ADAPT.** Do not install the dead package; the two mechanisms worth having — PostToolUse checksum ghost-edit detection and PostToolUse exit-code verification — are ~40 lines each, and re-implementing them in `constitution/scripts/hooks/` under our own §11.4.224 test-first discipline (with per-session/per-alias state dirs) yields the same binding without the dead dependency. This is precisely a SEAM mechanism our stack lacks (see §6).

. `skills` CLI (vercel-labs) + `howardng97/claude-stop-hallucination`

- The delivery channel is real and huge: npm `skills`, vercel-labs/skills, 87 versions, modified 2026-07-22 (the day of this verification), 51,903,010 downloads/month (npm downloads API, measured — reported as-is). `npx skills add <owner>/<repo>` installing from GitHub is the real usage shape.
- The skill itself: repo exists; 1 star, 0 forks, created and last pushed **the same day** (2026-04-29). Contents: `SKILL.md` (name `reality-check` — matching the transcript's uninstall note), `bin/install.mjs`, README. The SKILL.md frontmatter verifies the transcript's description: anti-sycophancy triggers ("reality check", "be real", "no sugarcoat", "brutal honesty"), allowed-tools Read/Grep/Glob/Bash.
- **Assessment:** it is a TONE INSTRUCTION — prose in context, the exact layer our research proved does not bind (Ontario null, corpus 2, 241/413 unimplemented gates). Our §11.4.6 no-guessing + §11.4.1 FAIL-bluff + the covenant's forbidden-vocabulary list are stronger and already propagation-gated. Installing third-party instructions into agent context from a 1-star repo is also an unnecessary prompt-surface. **REJECT — new vocabulary for an old rule, weaker than what it would sit beside.**

. Evidence tagging `[VERIFIED-REAL]` / `[VERIFIED-SYNTHETIC]` (the "sergeeey repo")

- The attribution resolves: GitHub user `sergeeey` exists; the source is `sergeeey/Claude-cod-top-2026` (7 stars, pushed 2026-07-22, description "Still trusting results that AI generated, tested, and declared successful by itself?…"). Its README carries the markers verbatim ("Every claim carries an evidence marker — `[VERIFIED-REAL]` (real data, sources cited) vs

`[VERIFIED-SYNTHETIC]` (mock data, never valid for production claims)…hard rule in `rules/integrity.md` "), plus an extended scale (`[VERIFIED-LOW]` etc.). Notably the repo is honest about its own maturity: "dogfood evidence still growing (2 real runs so far)".

- **The comparison the task demands (theirs vs ours):** ours is stronger where it is implemented, and their one genuine insight is already covered: (a) §11.4.69 requires a closed-set feature class + an existing non-empty artefact path per PASS — a machine-checkable precondition, vs a free-text marker the agent simply types (a self-applied marker is exactly as forgeable as the "done" claim it decorates); (b) the synthetic-vs-real discrimination — their genuinely explicit contribution — is our §11.4.27 (mocks/fakes ONLY in unit tests; every other test type against the real system) plus §11.4.108 layers 3–4, and it is being mechanized as **SOL-04 evidence-class-at-closure** (runtime-vs-source class recorded and enforced at the status-write seam), which BINDS where their marker only labels. (c) Their marker does have one ergonomic virtue: it is inline and cheap. If wanted, that virtue lands as a field in SOL-04's verdict files, not as CLAUDE.md prose. **REJECT as prose; the concept is already ours and already being landed bindingly (SOL-04).**

. `darcyegb/ClaudeCodeAgents` (Jenny / Karen / TaskCompletionValidator / …)

- **Exists:** 748 stars, 69 forks, created 2025-07-31, last push 2026-05-23, default branch `master` (the transcript's implied `main` clone works anyway; the tree fetch required `master`). All six transcript-named agents verified present as files: `Jenny.md` , `karen.md` , `task-completion-validator.md` , `claude-md-compliance-checker.md` , `code-quality-pragmatist.md` , `ui-comprehensive-tester.md` , plus `ultrathink-debugger.md` .
- **License: NONE** — GitHub reports no license; under copyright default that is all-rights- reserved, making "clone into `.claude/agents/` and ship it in your workflow" legally unclean for a commercial project regardless of stars.
- **Capability vs ours:** these are prompt-defined reviewer personas. Our review layer is already MANDATORY (§11.4.142 every change, no exception), independent (§11.4.70/§11.4.20), iterated to zero-finding GO (§11.4.134), exhaustive-scenario (§11.4.194), and PINNED to Fable-xhigh (§11.4.209). "Karen gives a 0–10 reality score" (file presence verified; scoring rubric inside

the file not deep-read — minor `UNKNOWN:`) and "TaskCompletionValidator score ≥ 8/10" are VIBE-SCORE gates — strictly weaker than our captured-evidence-or-no-GO requirement: a numeric self-assessment by a model is not an artefact and cannot satisfy §11.4.5/§11.4.69. **REJECT: our anchors are stronger on every axis these agents cover; the only thing they add is a lower bar with a friendlier name — plus a license problem.**

[4] [5]

. `fivetaku/fablize`

- **Exists + alive:** 870 stars, 117 forks, MIT, created 2026-06-14, pushed 2026-07-06, language Python. Tree (48 entries) verified: `.claude-plugin/{plugin,marketplace}.json`, `hooks/{gate_prompt.py, gate_post_tool.py, gate_stop.py, finish-the-work.sh, router.sh, hooks.json}`, `scripts/gate/{classify_task.py, ledger.py, parse_tool_result.py, verify_state.py}`, `scripts/goals.py`, shadow-measurement scripts, `setup/{setup.sh, uninstall.sh}`, `docs/MEASUREMENT_PROTOCOL.md`, and **six test files** under `tests/`.
- **Does what is claimed — YES, unusually so.** README (fetched raw) confirms, in its own words, each transcript claim: verification grounding ("render/executable artifacts … are run and observed before completion"), "Multi-story verification gate — `goals.py` decomposes work and refuses a groundless 'done'", "Early-stop hook … catches 'I'll do X' without doing it" (deterministic hook), escalate-at-capability-ceiling ("fablize tells you to escalate instead of pretending"), the exact install commands the transcript gives (`/plugin marketplace add fivetaku/fablize`, `bash ${CLAUDE_PLUGIN_ROOT}/setup/setup.sh` choose local/global), and an honest limitations section (the early-stop hook "can misfire on a declarative offer"). The transcript's "shows you a screenshot in the chat" is a paraphrase of run-and-observe, slightly overstated but directionally right. The transferable/non-transferable table in the transcript matches the README's shipped/not-possible table.
- **Relevance nuance for THIS project:** fablize's premise is making OPUS behave like Fable. Our reviews are already Fable-pinned (§11.4.209). But our WORKER agents run across a multi-alias fleet on varying substrates — and fablize's load-bearing mechanism is substrate-independent: a **Stop-hook completion gate** (`gate_stop.py`) that refuses session completion without

verification evidence, backed by a state ledger. That is a mechanism AT A SEAM — the class our central finding says we lack (prose does not bind; seams do), at a seam (Stop) where we currently run NOTHING (see §6).

- **Supply-chain:** MIT; auditable size; Python hooks executing with our privileges on every prompt/tool/stop event; has its own tests and a written measurement protocol. Risk is real but manageable via read-before-run audit. Installing it is NOT a reversible-low-blast-radius autonomous call (§11.4.101) — operator decision.
- **Verdict: ADAPT** — two admissible paths for the conductor/operator: (a) sandbox-trial the plugin as-is in one isolated track worktree with the hooks read first; or (b) treat `gate_stop.py` + ledger as a reference design and land our OWN §11.4-aligned Stop-gate (deny completion while the session's touched work items lack class-matched evidence artifacts — which would wire directly into SOL-01/SOL-04's custody chain at a seam none of SOL-01..10 currently occupies). Path (b) is the recommendation: it converts the best idea in the operator material into owned, tested, license-clean seam code.

. `anthropics/claude-plugins-official` + `mcp-server-dev`

- Marketplace repo: official Anthropic, 32,478 stars, Apache-2.0, pushed 2026-07-22. The specific plugin claim is proven by RUNTIME evidence stronger than any registry listing: this very session's live skill list contains `mcp-server-dev:build-mcp-server`, `mcp-server-dev:build-mcp-app`, `mcp-server-dev:build-mcpb` — the plugin exists AND is already installed in this environment. The transcript's scaffolding advice is real. No action needed.

. `@fabriciofs/mcp-postgres`

- Exists (v1.0.1, 2026-01-22, 2 versions same day); GitHub `fabriciofs/mcp-postgres` exists (1 star, pushed 2026-02-05). All five transcript-claimed tool names verified PRESENT in the registry README (`sql_execute`, `schema_list_tables`, `schema_describe_table`, `monitor_active_queries`, `analyze_query`). 53 downloads/month. Note the README also advertises WRITE operations behind `READONLY=false` — the transcript's read-only framing is a configuration, not a property.

- **Fit here: none.** This project has no PostgreSQL. Its authoritative runtime states are the DEVICE (live ADB — §11.4.51 already mandates maximizing it), the sink-side APIs (§11.4.13), and the workable-items SQLite DB (§11.4.93/95, queried directly). The transcript's underlying point — "verified" must mean a live query against real state — is our §11.4.108 RUNTIME layer verbatim. **REJECT the package; the principle is already ours** (and where it is not enforced, that is SOL-01/SOL-04's job, not an MCP server's).

. `@modelcontextprotocol/server-github`

- Exists but the npm latest (2025.4.8) carries the registry deprecation notice **"Package no longer supported."** (573,145 dl/month of legacy traffic notwithstanding). Recommending it in 2026-07 is stale advice; the maintained path is GitHub's own server or — in this repo — the `gh` CLI we already use. **REJECT.**

. `claude-pty-acp` + ACP

- npm `claude-pty-acp` exists: v0.1.0, created 2026-06-13, 1 version, 40 dl/month, repository field pointing at `github.com/ClayCheung/claude-pty-acp` — which returns **HTTP 404** (deleted or made private). The transcript's primary install path (`git clone …ClayCheung/claude-pty-acp.git`) **fails today**, and the npm tarball is an un-reviewable published artifact whose source is gone. That combination — install-and-execute a package whose source cannot be read — is a textbook supply-chain refusal. **REJECT outright.**
- ACP itself is real and healthy: the official Zed adapter `@zed-industries/claude-code-acp` (v0.16.2, 67,931 dl/month, modified 2026-03-26) exists, and the JetBrains claims in the transcript verify (native ACP "bring your own agent" 2025-12; ACP Agent Registry live 2026-01; Settings | Tools | AI Assistant | Agents install path — JetBrains official blog + help pages, accessed 2026-07-22). The `zed-industries/agent-client-protocol` repo itself could not be re-fetched (GitHub API 403 rate-limit at that moment — `UNKNOWN:` at the direct-fetch level, existence evidenced via the official ecosystem pages above).

- **Fit here:** the transcript positions ACP as "isolated verification contexts". We already have that capability natively: subagent context isolation (§11.4.20/ §11.4.70) and the §11.4.187 headless spawn primitive (`claude -p --output-format stream-json` with per-alias env) give fresh, memory-bias-free verifier instances without any adapter. ACP buys editor interop (Zed/JetBrains/Neovim), not verification power. **REJECT for the stated need**; keep the official Zed adapter in mind only if editor interop is ever wanted.

. What we already have — capability map (new mechanism vs new vocabulary)

The honest question is not "should we adopt this" but "what does it give us that our anchors do not — and does it BIND where ours only PRESCRIBE". Mapping every transcript recommendation:

Transcript recommend-ation	Our existing coverage	Which is stronger?	Genuine gap?
Evidence tagging `[VERIFIED-REAL]` / `[VERIFIED-SYNTHETIC]` in CLAUDE.md	§11.4.5 + §11.4.69 (closed-set taxonomy + required artefact path per class, `ab_pass_with_evidence`), §11.4.27 (no fakes beyond unit tests), SOL-04 (evidence-class-at-closure, mechanized)	**Ours** — theirs is a self-applied free-text marker (forgeable prose); ours requires an existing non-empty artefact and SOL-04 enforces the runtime-vs-source class at the status-write seam	No new capability. Their one explicit virtue (inline synthetic-vs-real label) lands as a SOL-04 verdict-file field, not prose
PreToolUse/ PostToolUse hooks (TruthGuard)	§11.4.109 PreToolUse guards WIRED AND FIRING here (see §7 local evidence) — but **only Pre**; we run ZERO PostToolUse hooks	**Split.** Preventive layer: ours (broader, tested, fleet-propagated). Detective layer: **theirs — we have no PostToolUse verification at all**	**YES — PostToolUse ghost-edit checksum + exit-code verification is a real, cheap, unoccupied seam** (≈80 lines, re-implement in-house per §4.1)

Transcript recommend-ation	Our existing coverage	Which is stronger?	Genuine gap?
QA agents (Jenny / Kar-en / TaskComple-tionValidator)	§11.4.125 review gate before build, §11.4.142 every-change review, §11.4.134 it-erate-until-GO, §11.4.194 all-scenario, §11.4.209 Fable-xhigh pin	**Ours, decis-ively** — theirs terminate in a model-emitted 0–10 vibe score; ours ter-minate in cap-tured evidence + zero-finding GO	No. (And theirs is unli-censed.)
Anti-syco-phancy / real-ity-check skill	§11.4.6 no-guessing (closed forbidden vocabulary, propagation-gated), §11.4.1 FAIL-bluff	**Ours** — same intent, ours is enforceable text with gates; theirs is a tone toggle	No
"No evidence = not done" rule block	The §11.4 covenant itself + §11.4.123 + §11.4.108 defini-tion-of-done	**Ours** (theirs is a 500-token re-statement)	No

Transcript recommend-ation	Our existing coverage	Which is stronger?	Genuine gap?
MCP live DB/ staging quer-ies before "verified"	§11.4.108 RUNTIME layer, §11.4.13 sink-side evidence, §11.4.51 live-ADB-first, direct SQLite on the workable-items DB	**Ours in principle — but note the honest self-as-sessment:** these are PRE-SCRIPTIONS; the binding ver-sions are SOL-01/ SOL-04 (in flight). An MCP Postgres server is object-level inapplicable (no Postgres); the device IS our staging DB and live-ADB is already first-class	The gap is enforcement, not tooling — already owned by SOL-01/04, not by any package here
ACP isolated verification contexts	§11.4.20/§11.4.70 subagent isolation + §11.4.187 head-less spawn (fresh context per verifier, per-alias env)	**Ours** — equi-valent isolation, zero new de-pendencies; ACP adds edit-or interop only	No

Transcript recommend-ation	Our existing coverage	Which is stronger?	Genuine gap?
Stop-gate / completion refusal (fablize)	**NOTHING at the Stop seam.** §11.4.87/§11.4.126 describe completion conditions as PROSE; no hook enforces them; SOL-01..10 sit at com-mit/build/status-write/release/ dispatch/intake — none at Stop	**Theirs — this seam is empty in our stack**	**YES — the second real gap** (see §6)

[6]

. The short list: things that BIND where our anchors only PRESCRIBE

The central finding of the whole research effort (solutions/README §1: "prose does not bind; seams do"; 241/413 named CM-* gates unimplemented; Ontario null; corpus-2 doctrine-without-hooks) is the axis the operator material must be judged on. Judged on exactly that axis, the transcript contains **two** things that are real, working, executing-at-a-seam, and not already in our stack or in SOL-01..SOL-10:

1. **PostToolUse verification hooks (TruthGuard's two detective scripts).** We currently wire PreToolUse (preventive: block forbidden commands / mislabeled dispatch) and UserPromptSubmit (intake) and SessionStart — and NOTHING detective after a tool runs. The ghost-edit checksum compare and the exit-code-vs-claim check execute mechanically on every Write/Edit/ Bash regardless of what the model believes, and feed a machine verdict back into the loop. This is a per-tool-call micro-instance of §11.4.108's running-artifact-equals-claimed-artifact rule, enforced at a seam we do not occupy. Cost to own: ~80 lines of stdin-JSON bash under `constitution/scripts/hooks/`, test-first per §11.4.224, with per-session state dirs (the upstream package's `~/.truthguard` global store is multi-alias-unsafe here, §4.1).
   Recommendation to the solutions agent: land as a SOL-class mechanism (suggested name: post-tool verification pair), citing spyrae/truthguard as the reference design (§11.4.8 external-source citation), NOT as a dependency.

2. **A Stop-seam completion gate (fablize's `gate_stop.py` + ledger concept).** Our done-conditions (§11.4.87(A), §11.4.126 terminal conditions, §11.4.197 completion mandate) are consulted by agents, not checked by any seam — the exact §11.4.205 test ("if I documented this and nobody implemented the hook, would anything catch it?" — today: nothing would). A Stop hook that refuses session completion while the session's claimed-done work lacks class-matched evidence artifacts converts the §11.4 covenant's weakest moment (the final "done" message) into a checked seam, and composes directly with SOL-01 (custody) and SOL-04 (evidence class): the Stop gate is the LAST-CHANCE consumer of the same verdict store. fablize proves the mechanism is buildable and live at 870 stars; the recommendation is path (b) of §4.5 — own implementation, fablize as cited reference design; a sandboxed one-track trial of the plugin itself is the acceptable faster variant IF the operator authorizes third-party hook execution after a read-through (not an autonomous call, §11.4.101/ §11.4.122-class).

Everything else in the transcript is either (a) new vocabulary for an anchor we already carry in stronger form, (b) object-level inapplicable (Postgres), (c) deprecated/dead/unauditable, or (d) already installed here (mcp-server-dev).

. Would-it-work-here notes (local runtime evidence)

- This checkout's `.claude/settings.json` (read-only inspection) wires: SessionStart (multitrack monitor), PreToolUse `Agent|Task|TaskCreate` (track-branch-label + work-track-binding guards), PreToolUse `Bash` (forbidden-commands + branch-consistency + work-track-binding guards), UserPromptSubmit (action-prefix expansion + operator-request capture). The guard scripts + their test files exist in `constitution/scripts/hooks/`. Live-fire runtime evidence: a PreToolUse hook visibly injected context on Bash calls during THIS session — hooks here are not just configured, they execute (§11.4.196(F) configured-vs-in-use satisfied for the Pre layer).
- Confirmed absent locally (the two gaps of §6): no `PostToolUse` key and no `Stop` key exist in the settings hooks object.

- Multi-alias layout caveat: any "global" install path in the transcript (`npx skills add … -g` , `~/.truthguard/` , `~/.claude/settings.json`) lands per-alias or per-home state that the `CLAUDE_CONFIG_DIR=.claude-<alias>` fleet would see divergently — per-project wiring under `$CLAUDE_PROJECT_DIR` (as our existing hooks already do) is the only layout that behaves uniformly across aliases and tracks.
- Rootless-podman (§11.4.161): none of the verified artifacts require containers; not a constraint for any verdict above.

. Was the operator-supplied material accurate, or confabulated? (the meta-finding)

Both — in a precisely instructive split, and the split is the finding:

Accurate (verified true, sometimes surprisingly): every named repo/package except one dead link exists; fablize's every specific behavioral claim matches its source; the ClaudeCodeAgents roster matches file-for-file; the `[VERIFIED-REAL]` rule genuinely traces to the "sergeeey" repo; TruthGuard's checksum/ghost-edit and exit-code mechanisms are real code; the "30 hook events" claim matches the current docs (~30–31 — a claim I initially expected to be inflated and which verified); the JetBrains/ACP timeline and settings path are right; the install command syntax for `skills` , plugins, and fablize is letter-accurate.

Confabulated (verified false): `npx truthguard audit` (no such command); `truthguard pre` / `truthguard post` subcommands (no such commands — and the Step-2 hook block built on them uses a non-existent flat-string schema); `truthguard.config.json` (no such file; real artifact is `.truthguard.yml.example` and hooks land in settings.json); `$CLAUDE_TOOL_INPUT` / `$TOOL_NAME` / `$TOOL_ARGS` env vars (not in the docs; the transcript's force-push-block hook would silently never block — a fabricated GUARD that is itself a §11.4.201 false-negative); `.claude/last-session.log` (no such path, so the "Ready-for-QA evidence scan" step greps a nonexistent file); `git clone …ClayCheung/claude-pty-acp.git` (404); and the recommendation of a package npm marks deprecated. Confidence-presentation failure: 2-star dead

projects and 870-star maintained ones are presented with identical authority ("battle-tested", "hundreds of hours of trial-and-error" — claims with no possible backing).

The irony is complete and worth recording: a guide whose thesis is "don't ask the agent to be honest — verify it" contains fabricated commands, a fabricated config file, a fabricated log path, and a verification hook that verifies nothing. Followed verbatim, steps 2 and the QA-gate evidence-scan would each install a silent no-op believed to be protection — the most dangerous class in our taxonomy (a blind instrument and a clean artifact return the identical quiet zero, §11.4.201(6)). The material simultaneously proves its own point: the two artifacts in it that survive scrutiny (fablize; the TruthGuard detective-hook pattern) are exactly the ones that execute at seams, and the parts that collapse are exactly the prose and the unverified specifics.

[3]

. Recommendations handed to the Phase solutions agent

Item	Verdict	Action owner
PostToolUse ghost-edit + exit-code verification pair (in-house, TruthGuard-referenced)	**ADAPT — build ours**	solutions agent: new SOL entry at the tool-call seam; then a tracked work item to wire (per §11.4.205 no force until wired)
Stop-seam completion gate (in-house, fablize-referenced; consumes SOL-01/04 verdict store)	**ADAPT — build ours** (sandbox trial of fablize itself = operator-gated alternative)	solutions agent + operator decision for any third-party trial
fablize plugin as-is	Operator-gated trial only (read hooks first; one isolated track)	conductor → operator (§11.4.101/§11.4.122-class)
mcp-server-dev / official marketplace	Already present here; no action	—
TruthGuard package, reality-check skill, ClaudeCodeAgents, @fabriciofs/mcp-postgres, @modelcontextprotocol/server-github, claude-pty-acp, evidence-tagging prose block	**REJECT** (reasons per §4)	—
ACP / official Zed adapter	Not needed for verification isolation; note for future editor-interop only	—

. Anti-bluff certification

- Every EXISTS/ABSENT/DEPRECATED claim above carries its check (registry JSON, GitHub API response, raw-file fetch, or live docs fetch) with the access date 2026-07-22; download counts are the npm downloads API window 2026-06-22 → 2026-07-21, reported verbatim.
- Both instrument paths were control-needled before any absence was reported (§1); the one rate-limited query was reported as 403, not as absence; per-query needles accompanied the two in-file absence claims (`audit` , `truthguard.config`).
- Honest `UNKNOWN:` markers stand on exactly three low-stakes points: `/reload-plugins` existence; letter-exact `claude mcp add --transport http` syntax; Karen's internal 0–10 rubric. Nothing else in this document is inferred without a cited check.
- Nothing was installed, cloned into `.claude/` , or executed from any third-party source; reads only. No governance file, settings file, hook, DB, or gate script was modified. This document is the session's only write.

Sources verified – –

- npm registry: `registry.npmjs.org/{express, truthguard, skills, claude-pty-acp, @fabriciofs/mcp-postgres, @modelcontextprotocol/server-github, @zed-industries/claude-code-acp}` + `api.npmjs.org/downloads/point/last-month/...`
- GitHub API/raw: `spyrae/truthguard` (+ full tree + `bin/truthguard.js` + `scripts/pre-file-change.sh` + `scripts/check-file-change.sh` + `hooks/hooks.json`), `spyrae/homebrew-truthguard` , `darcyegb/ClaudeCodeAgents` (tree@master), `fivetaku/fablize` (tree + README), `howardng97/claude-stop-hallucination` (tree + SKILL.md), `anthropics/claude-plugins-official` , `anthropics/claude-code` (needle), `fabriciofs/mcp-postgres` , `users/sergeeey` + repo list + `sergeeey/Claude-cod-top-2026` README
- Claude Code docs: code.claude.com/docs/en/hooks (hook-event list, stdin-JSON delivery, settings schema, env-var list)

- JetBrains/ACP: Bring your own AI agent to JetBrains IDEs (2025-12), ACP Agent Registry Is Live (2026-01), JetBrains ACP help, jetbrains.com/acp, Agents | AI Assistant Documentation
- Local runtime: this checkout's `.claude/settings.json`, `constitution/scripts/hooks/` listing, and this session's live skill list (mcp-server-dev presence) + live PreToolUse hook fire